

PQTrust-DAG: A Post-Quantum Behavioral Trust Layer for DAG-Based Vehicular Sensor Networks

L DILEEP SAI¹, AND UMA SANKARARAO VARRI¹

¹Department of Computer Science and Engineering, SRM University AP, Amaravathi, Andhra Pradesh, India (e-mail: dileep_lavu@srmap.edu.in; umasankararao.v@srmap.edu.in)

Corresponding author: L Dileep Sai (e-mail: dileep_lavu@srmap.edu.in).

This work did not receive any external funding.

ABSTRACT In a vehicular sensor network, a digital signature shows who sent a reading, but not whether the reading is true. A vehicle with valid keys can still be hacked, broken, or dishonest. This paper adds a behavioral trust layer to a post-quantum, DAG-based vehicular sensor network. A Trust Manager at each gateway gives every vehicle a trust score. The score decides whether the data of the vehicle is counted in proportion to its trust, counted at half of that, held back, or not used at all (quarantine). Each trust update is signed with ML-DSA-65 and stored in an audit ledger. We test the system in simulation with 1,000 sensors and 100 gateways, with no attack and with five attacks: false data, faulty sensors, on-off attacks, collusion, and bad-mouthing. Our method and the published systems are tuned on separate random seeds and tested on five new seeds. At the end of a run, the tuned trust score detects 99.7% of attackers and wrongly quarantines fewer than 0.1% of honest nodes. Two published reputation systems, rebuilt from their papers and tuned in the same way, detect slightly more attackers but wrongly quarantine 1.6% and 2.4% of honest nodes (with our evidence rule). Removing the history term cuts detection to 82.5%. Adding the peer-report or reliability terms lowered detection in our tests. Signing the 100,000 trust updates of one run took 83 to 88 minutes with a pure-Python library, and 81% of DAG transactions were finalized. All results come from simulation.

INDEX TERMS Blockchain, Directed Acyclic Graph, Post-Quantum Cryptography, Trust Management, Vehicular Sensor Networks

I. INTRODUCTION

INTELLIGENT Transportation Systems (ITS) use Vehicular Sensor Networks (VSNs) to make real-time decisions about traffic, road conditions, and safety. A VSN collects data from sensors on vehicles and on the roadside, such as GPS, LiDAR, radar, cameras, and inertial measurement units. It shares this data between vehicles (V2V), with roadside units (V2I), and with cloud servers (V2C). This data can affect collision avoidance and traffic routing. So the network must keep the data authentic, and it must also check that the data is correct.

Two threats make this hard. The first is quantum computing. Shor's algorithm [1] solves factoring and discrete logarithms efficiently on a quantum computer. A large quantum computer could therefore break the RSA and elliptic-curve schemes that current V2X systems use for authentication and key exchange. Grover's algorithm [2]

also weakens symmetric ciphers and hash functions, which is why post-quantum standards use longer keys and outputs. The second threat has nothing to do with cryptography. Vehicular networks are open, so a user with valid credentials can still send fake data, inject false readings, create fake (Sybil) identities, or launch denial-of-service attacks.

Blockchains make shared data hard to change. However, designs that add blocks one after another in a single chain scale poorly once the extra cost of post-quantum cryptography is added. DAG-based ledgers help, because each transaction points to several earlier transactions, so many transactions can be checked in parallel. DAGs have been proposed to improve blockchain scalability in general [3]–[6] and for post-quantum VSNs in particular. Ahmed et al. [7] describe such a design: a DAG-based VSN blockchain that uses CRYSTALS-Kyber for key encapsulation, CRYSTALS-Dilithium for signatures, a

lightweight post-quantum proof-of-work (PoPQW), and threshold signatures for finality. It moves heavy computation from the sensors to the gateway and server layers. A design like this protects identities and scales the ledger. It does not check whether authenticated data is true.

Research gap. The architecture in [7] shows, with quantum-safe cryptography, who sent a message. It does not show whether the message should be believed. Several problems pass through this gap even when every signature is valid:

- a vehicle with a valid key sends fake events, such as a fake accident, a wrong traffic count, or a false location;
- an attacker takes over a real vehicle and sends bad data with its valid keys;
- a broken or badly calibrated sensor keeps signing and sending wrong readings;
- a roadside unit or fleet operator with insider access changes data after it has been authenticated;
- several authenticated nodes work together to bias a shared reading or a vote.

In all these cases, the cryptography works as intended. The open question is whether the content of an authenticated message can be trusted. This paper asks two questions. Can a behavioral trust layer, tested apart from the post-quantum and DAG parts, tell good data from bad data? How does it compare with well-known reputation systems that are tuned in the same way?

Our approach. We add a behavioral trust layer to the architecture of [7] and leave its cryptography unchanged. A Trust Manager at each gateway does four things. First, it gives every node a trust score from four signals: how reliably the node transmits, how consistent its readings are, how it behaved in the past, and what other nodes report about it. Second, it uses the score to put the node in one of four tiers (Trusted, Cautious, Suspicious, Quarantined). Third, it uses the tiers to decide how much each reading counts when the gateway combines the readings of its sensors. Fourth, it writes a signed, hashed record of every trust update to a separate audit ledger, *BC_Trust*. Section III-C lists which parts of the design we built and which we did not.

What we found. In simulation, the trust score that works best combines a smoothed consistency check with a long memory of the node's own past trust. At the end of a run, it detects 99.7% of attackers and wrongly quarantines fewer than 0.1% of honest nodes. Two well-known reputation systems [41], [42], rebuilt from their papers and tuned in the same way, detect slightly more attackers (99.9% and 100%). But, with our evidence rule, they quarantine 1.6% and 2.4% of honest nodes at the end of a run, and 97% and 100% of honest nodes at some point during the run. We also report results that do not favor our design. Adding the peer-report or reliability terms lowered detection in our tests. Almost

no honest node reaches the top tier. A simple unweighted baseline gives a smaller aggregation error.

A. CONTRIBUTIONS

- A trust model with graded responses. A trust score from four signals (reliability, consistency, history, and peer reports) drives a four-tier policy with hysteresis, and the tiers set the weight of each reading in the aggregate. We show which parts of the score carry the result and which do not.
- A fair test protocol. Our method, BRS, and RRS are tuned with the same objective on tuning seeds 0–4 and then tested on five different seeds (42–46). The trust manager never sees the true labels of the nodes. We report two detection measures: whether a node was ever quarantined, and whether it is quarantined at the end of the run.
- A comparison with two published reputation systems. We rebuilt the Beta Reputation System [41] and the robust reputation system of Buchegger and Le Boudec [42] from their equations, gave them the same evidence, and tuned them with the same objective.
- An ablation study with negative results. We remove or add one part at a time. The history term and the consistency term carry the result. Adding the peer-report or reliability terms lowers detection.
- A working, timed audit ledger. *BC_Trust* stores trust records signed with ML-DSA-65. We ran all six scenarios end to end with signing and DAG commits (100,000 signatures per run) and measured the cost.

B. ORGANIZATION

Section II reviews related work. Section III describes the architecture and what we built. Section IV explains the trust model and its algorithms. Section V gives the experimental setup and the results. Section VI discusses what the results mean and where they stop. Section VII concludes the paper and lists future work.

II. RELATED WORK

A. BLOCKCHAIN FOR VEHICULAR NETWORKS

Early blockchain systems for vehicles inherited the scaling limits of their chains. Xia et al. [8] used Proof-of-Work (PoW) to share medical records, which is a poor fit for time-critical vehicular networks. They later applied the same trust-less sharing idea among cloud service providers [9]. Kang et al. [10] combined a consortium blockchain with a reputation scheme based on subjective logic for data sharing in vehicular edge computing, but their data auditing relies on a PoW-style process. Zhang and Chen [11] used a consortium chain with Byzantine fault-tolerant (BFT) consensus for VANETs. This is better than PoW but slows down as the number of validators grows. Fan et al. [12], Cui et al. [13],

Rahman et al. [14], Vieira et al. [15], de Anda-Suárez et al. [16], and Das et al. [17] propose other schemes for data sharing, consensus and trading, or identity management in vehicular networks. None of them uses post-quantum cryptography or a behavioral trust layer. Zrikem et al. [18] propose a Vehicle-to-Blockchain (V2B) architecture on public blockchains, which includes a vehicle-scoring system based on how well drivers follow traffic rules. Bendiab et al. [19] survey how blockchain and artificial intelligence can be combined to secure autonomous vehicles. Xie et al. [20] propose a provably secure, lightweight blockchain protocol for V2I handover and V2V broadcast authentication, based on elliptic-curve cryptography.

DAG-based ledgers remove the single-chain bottleneck, because each transaction is checked in parallel against several earlier transactions. Raikwar et al. [5] survey DAG-based consensus protocols. Mahdi and Rabee [6] propose an improved DAG structure, and Shi et al. [21] model and test the performance of DAG-based ledgers. Hassija et al. [22] use a DAG for data sharing and energy trading between vehicles and the power grid (V2G), and report high scalability for micro-transactions. Liao et al. [3] propose IORT-DAG, a real-time DAG with implicit ordering. Chai et al. [23] propose a DAG blockchain for knowledge sharing among connected vehicles, with a lightweight tip-selection algorithm for consensus and identity checks. None of these DAG designs deals with post-quantum security or behavioral trust.

B. POST-QUANTUM CRYPTOGRAPHY FOR VEHICULAR SYSTEMS

NIST has standardized three post-quantum schemes: ML-KEM for key encapsulation (FIPS 203, derived from CRYSTALS-Kyber), ML-DSA for signatures (FIPS 204, derived from CRYSTALS-Dilithium), and SLH-DSA, a stateless hash-based signature (FIPS 205, derived from SPHINCS+) [24]–[26]. Its fourth-round status report selects HQC as an additional key-encapsulation scheme [27]. Several recent works use such schemes in vehicular settings. Xu et al. [28] combine post-quantum linkable ring signatures with a key-exchange mechanism on a blockchain. This gives vehicle authentication that is anonymous, traceable, and resistant to Sybil attacks. Son et al. [29] design a lightweight authentication scheme for UAV-enabled intelligent transportation systems using blockchain and physical unclonable functions (PUFs), but it relies on classical elliptic-curve cryptography. Chauhdary et al. [30] propose a lightweight PUF-based two-way authentication scheme for the Internet of Vehicles. It is built on an improved logistic map and does not use post-quantum primitives. Chen et al. [31] build a post-quantum blockchain for smart cities, with a PoW consensus that supports memory mining and identity-based post-quantum signatures for lightweight transac-

tions. Sandanamudi et al. [32] compare the performance of several post-quantum schemes for UAV communication, a resource-limited setting similar to VSNs. Chiang et al. [33] build post-quantum threshold ring signatures. These are relevant to the threshold-signature finality of [7], but we do not test them here. These works protect authentication, key exchange, or the ledger itself. None of them checks whether authenticated data is correct.

C. TRUST MANAGEMENT IN VEHICULAR NETWORKS

Reputation and trust scoring for VANETs is a well-studied topic [34]–[36]. Liu et al. [37] survey blockchain-based trust management for IoT. They explain that decentralization, consistency, and tamper-proofing help fix the weaknesses of centralized and distributed trust systems. However, most published trust models were designed without post-quantum or DAG assumptions and are not linked to a tamper-resistant ledger. Many check data consistency only loosely or not at all. Many also accept peer reports as they are, instead of weighting them by the reporter's own trust, which makes them open to bad-mouthing and collusion. Malik et al. [38] propose TrustChain, a blockchain-based trust and reputation framework for IoT-supported supply chains, but it uses a conventional chain and classical cryptography. Ahmad et al. [39] use quantum reservoir computing to detect intrusions from patterns in network and sensor data at the edge; they do not weight data by the trust of its source. Fu et al. [40] propose IoV-BERT-IDS, a hybrid network intrusion detection system for IoV based on a large language model. It also focuses on intrusion detection, not on trust-weighted data validation. None of these works uses a DAG ledger.

Two classic reputation systems are not specific to vehicles, but many later trust models build on them. The Beta Reputation System (BRS) of Jøsang and Ismail [41] keeps counts of good and bad observations for each node and turns them into a score with the beta distribution. It discounts advice from other nodes by their own reputation, and it forgets old observations with a forgetting factor. The robust reputation system (RRS) of Buchegger and Le Boudec [42] uses both first-hand and second-hand information. It checks whether a second-hand report agrees with the current rating, and it lowers its trust in nodes whose reports do not agree. Neither paper says how a good or bad observation is made from sensor data; both take it as an input. We use both systems as baselines in Section V-D.

D. POSITIONING

To our knowledge, no earlier work combines all three of the following: (i) post-quantum authentication of identities, (ii) a DAG-based ledger, and (iii) a behavioral trust score with several signals that directly controls which data is accepted and how it is combined. PQTrust-DAG targets this combination. Table 1 compares its design

with related systems. Because trust is kept separate from identity and consensus, each layer can be tested on its own. All trust results in Section V were obtained without running the consensus layer.

III. SYSTEM ARCHITECTURE

A. NETWORK MODEL

PQTrust-DAG has three tiers: a sensor layer, a gateway layer, and a server layer.

- 1) Sensor Node Layer: vehicles and roadside sensors with little computing power. They only sign and encrypt their own readings.
- 2) Gateway Layer (RSU): roadside units or edge servers. They authenticate and decrypt sensor reports and build DAG transactions. In PQTrust-DAG, they also compute trust scores and combine data using trust.
- 3) Server Layer: cloud or back-end servers that keep three ledgers: BC_ID for identity registration, BC_Data for DAG-based data, and BC_Trust for trust records.

Figure 1 shows the three layers, the data flow, and which ledgers we implemented.

B. LEDGER STRUCTURE

1) BC_ID

A permissioned identity ledger. Server nodes record the IDs of registered sensor nodes in it. Gateways and servers can read it, only server nodes can add blocks, and sensor nodes cannot access it [7].

2) BC_Data

The DAG-based data ledger from [7]. Each transaction points to earlier (parent) transactions. It carries the combined reading and, in our extension, a quality score.

3) BC_Trust (proposed)

A separate, lighter ledger for trust records. Each entry holds a node ID, a hash of the trust value, a hash of the supporting evidence, a time stamp, and a gateway signature. Raw behavior data stays off the chain, but every trust decision can still be audited later.

C. IMPLEMENTATION SCOPE

Our simulation (Section V) implements the trust layer: trust scoring, the four-tier policy, trust-weighted data combination, real transactions committed to a DAG with in-degree finalization, and BC_Trust records signed with ML-DSA-65. Six parts of the design are not implemented:

- 1) the PoPQW consensus and threshold-signature finality of BC_Data [7], so we do not measure ledger throughput;
- 2) the extended transaction format of Section IV-D;

- 3) the time stamp and replay check, because the simulation has no network delays or replayed packets (in the simulation, a reading is invalid only if it is outside the physical range);
- 4) signatures on peer reports, so the simulation models only the votes;
- 5) changing pseudonyms, because node IDs are fixed;
- 6) differential privacy for published trust statistics.

BC_ID is taken from [7] and is not simulated. The evidence hash in each BC_Trust entry is computed over an empty string in the simulation, so it shows the entry format and the signing cost only.

D. THREAT MODEL

The design targets six threats:

- 1) authenticated but malicious vehicles that send false readings;
- 2) sensors that fail or lose calibration;
- 3) groups of authenticated nodes that collude;
- 4) on-off attackers that switch between honest and malicious behavior to avoid detection;
- 5) nodes that falsely accuse honest nodes (bad-mouthing);
- 6) fake (Sybil) identities, which we assume the permissioned identity ledger blocks.

We assume that server nodes are not compromised, that gateways are honest, that the post-quantum primitives are secure, and that the network is eventually reliable. The simulation tests threats 1–5. Sybil identities depend on the identity layer and are discussed in Section VI-E.

IV. TRUST MANAGEMENT METHODOLOGY

This section explains how a gateway computes the trust score, how the score sets the tier of a node, and how the tiers set the weight of each reading. Figure 2 gives an overview, and Table 2 lists the symbols.

A. TRUST SCORE

Every node i has a trust score $T_i(t) \in [0, 1]$ that is updated once per round as a weighted sum of four signals (Eq. 1):

$$T_i(t) = w_R R_i(t) + w_C C_i(t) + w_H H_i(t) + w_M M_i(t), \quad (1)$$

with $w_R + w_C + w_H + w_M = 1$. Every node starts with $T_i(0) = 0.5$. The weights are chosen by a search (Section V-C).

1) Reliability

The reliability $R_i(t)$ (Eq. 2) is the share of valid transmissions among the last 100 transmissions of the node:

$$R_i(t) = \frac{\text{valid transmissions in the window}}{\text{transmissions in the window}}. \quad (2)$$

In the simulation, a transmission is invalid if its reading is outside $[0, 120]$ km/h. A full system would also count failed signature checks.

TABLE 1. Design-level feature comparison with related architectures (Section III-C lists what is evaluated in this paper)

Feature	Xia et al. [8]	Kang et al. [10]	Zhang et al. [11]	Malik et al. [38]	Ahmad et al. [39]	Base [7]	PQTrust-DAG
Post-Quantum Security	×	×	×	×	×	✓	✓ [†]
DAG-Based Architecture	×	×	×	×	×	✓	✓*
Behavioral Trust Mgmt.	×	‘partial’	×	✓	×	×	✓*
False Data Detection	×	×	×	×	‘partial’	×	✓*
Trust-Weighted Aggregation	×	×	×	×	×	×	✓*
Auditable Trust Ledger	×	×	×	✓	×	×	✓*

*Implemented and evaluated in this paper (Section V). [†]In this paper, only the ML-DSA signing of trust records is implemented. Other security guarantees come from [7] and the NIST standards. We measure the signing cost (Section V-K) but do not re-derive any security proof.

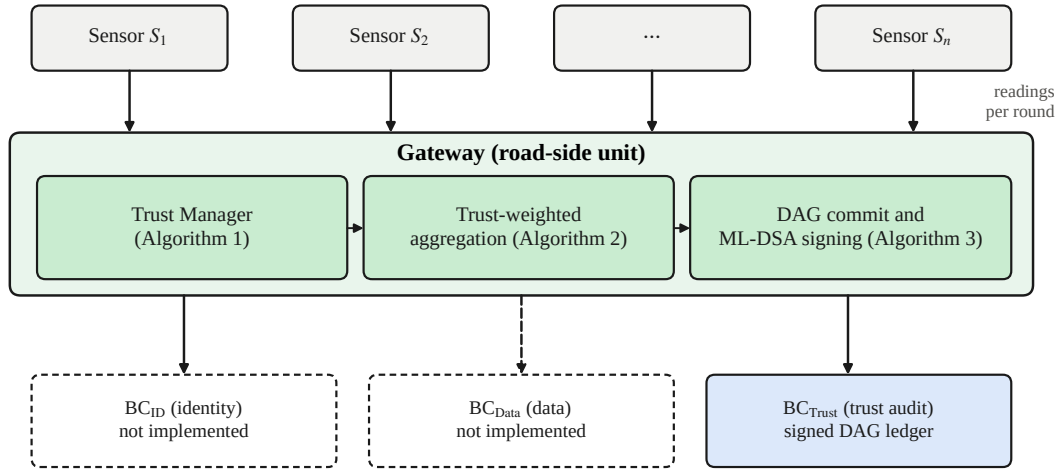


FIGURE 1. System model. Sensors send readings to their nearest gateway. The gateway computes trust (Algorithm 1), combines the readings (Algorithm 2), and signs and commits the result (Algorithm 3). Solid boxes are implemented in this paper. Dashed boxes are taken from [7] and are not implemented here (Section III-C).

TABLE 2. Notation

Symbol	Meaning
$T_i(t)$	Trust score of node i at round t , in $[0, 1]$
$R_i(t)$	Reliability signal
$C_i(t)$	Consistency signal
$H_i(t)$	History signal
$M_i(t)$	Peer-report signal
w_R, w_C, w_H, w_M	Signal weights; they sum to 1
k_1, k_2, k_3	Weights of the three consistency checks
$s_{temp}, s_{spatial}$	Scales of temporal and spatial checks
$x_i(t)$	Reading of node i at round t
m	Memory parameter of the history signal
v_r	Vote of observer r (1 = accuses, 0 = does not)
z	Small constant in the peer-report signal
b_R, b_Q, b_A	Quarantine, Cautious, Trusted limits
ω_i	Weight of node i in the aggregate
D_j, Q_j	Aggregate and quality score at gateway j
$DR, FPR, F1$	Detection rate, false-positive rate, F1-score

2) Consistency

The consistency $C_i(t)$ (Eq. 3) combines three checks:

$$C_i(t) = k_1 C_i^{temp}(t) + k_2 C_i^{spatial}(t) + k_3 C_i^{physical}(t), \quad (3)$$

with $(k_1, k_2, k_3) = (0.3, 0.4, 0.3)$. The temporal check (Eq. 4) penalizes a large jump from the previous reading of the same node:

$$C_i^{temp}(t) = \exp\left(-\frac{|x_i(t) - x_i(t-1)|}{s_{temp}}\right). \quad (4)$$

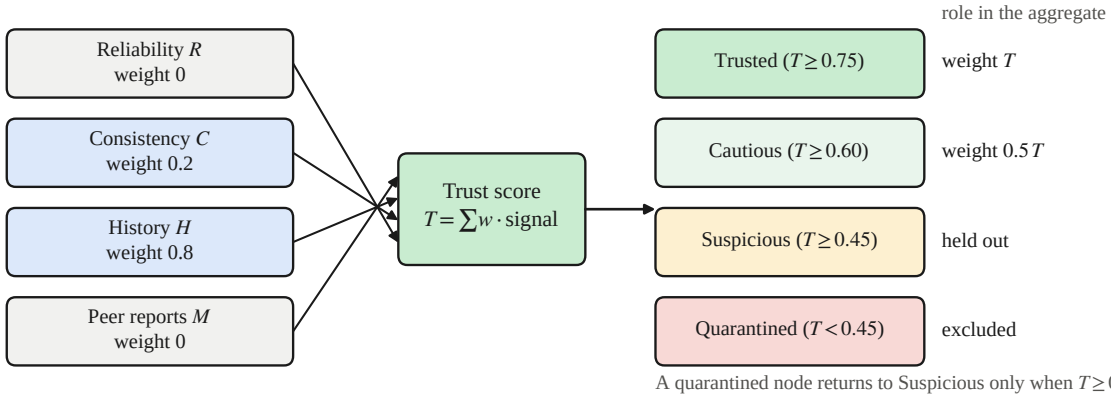
The spatial check (Eq. 5) compares the reading with $\bar{x}_i(t-1)$, the average of the previous-round readings of five randomly chosen neighbors at the same gateway:

$$C_i^{spatial}(t) = \exp\left(-\frac{|x_i(t) - \bar{x}_i(t-1)|}{s_{spatial}}\right). \quad (5)$$

The physical check (Eq. 6) is 1 inside the physical range and falls linearly outside it:

$$C_i^{physical}(t) = \begin{cases} 1, & x_i(t) \in [0, 120], \\ \max(0, 1 - \frac{|x_i(t) - 60|}{100}), & \text{else.} \end{cases} \quad (6)$$

If a node has no previous reading, or has no neighbors, the temporal or spatial check is set to 1. We use $s_{temp} = 20$ and $s_{spatial} = 15$. These scales were not tuned (Section V-C).



Algorithm 1 Trust update and tier decision

```

1: Input:  $x_i$ , last reading, neighbor readings, votes,  $T_i(t-1)$ 
2: Output: updated  $T_i(t)$  and tier
3:  $\text{valid} \leftarrow (0 \leq x_i \leq 120)$ 
4: compute  $R_i, C_i, H_i, M_i$  (Eqs. 2–8)
5:  $T_i \leftarrow \min(1, \max(0, w_R R_i + w_C C_i + w_H H_i + w_M M_i))$ 

6: if  $T_i < b_R$  then
7:   add  $i$  to the quarantine set
8: end if
9: if  $i$  is in the quarantine set and  $T_i \geq b_Q$  then
10:  remove  $i$  from the quarantine set
11: end if
12: if  $i$  is in the quarantine set then
13:  tier  $\leftarrow$  Quarantined
14: else if  $T_i \geq b_A$  then
15:  tier  $\leftarrow$  Trusted
16: else if  $T_i \geq b_Q$  then
17:  tier  $\leftarrow$  Cautious
18: else
19:  tier  $\leftarrow$  Suspicious
20: end if
21: return  $T_i$ , tier

```

Algorithm 1 gives one node a trust score and a tier in each round. Lines 3–5 compute the four signals and combine them into the score. Lines 6–11 apply the hysteresis rule: a node enters quarantine below $b_R = 0.45$ and leaves it only at $b_Q = 0.60$ or higher. Lines 12–20 set the tier from the score (Trusted from 0.75, Cautious from 0.60, otherwise Suspicious).

Algorithm 2 Trust-weighted aggregation

```

1: Input: readings  $\{x_i\}$  and tiers of the  $n$  sensors
2: Output: aggregate  $D_j$ , quality score  $Q_j$ 
3:  $S \leftarrow 0, W \leftarrow 0, \text{held} \leftarrow \emptyset$ 
4: for each sensor  $i$  do
5:   if tier of  $i$  is Trusted then
6:      $S \leftarrow S + T_i x_i; W \leftarrow W + T_i$ 
7:   else if tier of  $i$  is Cautious then
8:      $S \leftarrow S + 0.5 T_i x_i; W \leftarrow W + 0.5 T_i$ 
9:   else if tier of  $i$  is Suspicious then
10:    add  $i$  to held
11:   end if
12: end for
13: if corroboration is on and  $W > 0$  then
14:  core  $\leftarrow S/W$ 
15:  add  $i \in \text{held}$  if  $|x_i - \text{core}| \leq 20$ , weight  $0.5 T_i$ 
16: end if
17: if  $W > 0$  then
18:   $D_j \leftarrow S/W; Q_j \leftarrow W/n$ 
19: else
20:   $D_j \leftarrow$  median of non-quarantined  $x_i; Q_j \leftarrow 0$ 
21: end if
22: return  $D_j, Q_j$ 

```

Algorithm 2 combines the readings at one gateway. Lines 4–12 give each reading a weight by tier: Trusted counts fully, Cautious counts half, Suspicious is held out, and Quarantined is skipped. Lines 13–16 optionally let a held-out reading in if it is within 20 of the group aggregate. Lines 17–22 return the weighted mean and the quality score Q_j , or the median of the non-quarantined readings if no node has weight.

Algorithm 3 Signed trust record in BC_Trust

```

1: Input: node ID  $PD_i$ , new trust  $T_i(t)$ , evidence  $E_i$ , time  $t_{so}$ 
2: Output: BC_Trust entry
3:  $H_{\text{trust}} \leftarrow \text{SHA-256}(T_i(t) \parallel t_{so}); H_{\text{evid}} \leftarrow \text{SHA-256}(E_i)$ 
4:  $\text{msg} \leftarrow PD_i \parallel H_{\text{trust}} \parallel H_{\text{evid}} \parallel t_{so}$ 
5:  $\sigma \leftarrow \text{ML-DSA-65.Sign}(\text{sk}_{\text{gateway}}, \text{msg})$ 
6: append  $\{PD_i, H_{\text{trust}}, H_{\text{evid}}, t_{so}, \sigma\}$  to BC_Trust
7: return the entry

```

Algorithm 3 writes one signed entry to BC_Trust. Line 3 hashes the trust score and the evidence, so no raw data is stored (the simulation hashes an empty string as evidence). Lines 4–6 sign the node ID, the hashes, and the time with ML-DSA-65 and append the entry. Because the node ID and the time are signed, an entry cannot be moved to another node or date without breaking the signature (Section V-K).

V. EXPERIMENTS AND RESULTS**A. SETUP**

We built the simulation in Python 3 with NumPy. It contains the trust manager, the DAG ledger, and the BC_Trust ledger. The main tests use 1,000 sensors and 100 gateways in a 1000×1000 m area, with 15% malicious nodes and 100 rounds. Each sensor connects to its nearest gateway. One trust manager keeps the state of all nodes, and one ledger key signs all trust records. This gives the same outcome as one manager and one key per gateway, because each sensor belongs to one gateway. Each round, every node sends one reading. We test six scenarios:

- 1) No attack: every node reports a speed drawn from $\mathcal{N}(60, 15^2)$ km/h plus noise $\mathcal{N}(0, 3^2)$. The 15% of nodes that are labeled malicious behave honestly. This scenario measures false positives when there is no attacker.
- 2) False data: the malicious nodes report a speed drawn uniformly from $[20, 200]$ km/h in every round.
- 3) Faulty sensor: the malicious nodes send an honest speed draw plus noise $\mathcal{N}(0, 50^2)$.
- 4) On-off: the malicious nodes are honest for the first 59 rounds, send false data in the next 40 rounds, and send one honest reading in the last round.

- 5) Collusion: the malicious nodes all send the same false value in each round. The value depends only on the round number.
- 6) Bad-mouthing: the malicious nodes report honest data. Their attack is in the votes they cast as observers (see below).

Peer votes. Observers are drawn at random from all sensors at the gateway. An honest observer votes 1 if the reading differs by more than 40 from the average reading of the other sensors in the previous round, and 0 otherwise. In the collusion scenario, malicious observers never accuse malicious nodes and vote like honest observers on honest nodes. In the bad-mouthing scenario, malicious observers never accuse malicious nodes and always accuse honest nodes. In the other attacks, malicious observers vote like honest ones. The attackers know who their partners are, as part of the attack model, but the trust manager never sees the true labels.

Each gateway commits one Transaction per round to a shared DAG, so a full run creates 10,000 transactions. At the start of each round, the simulation takes a snapshot of all transactions that no other transaction points to yet (the tips). Each gateway picks up to three parents at random from this snapshot. A transaction is finalized when two later transactions point to it. Each trust update is signed with ML-DSA-65 using the pure-Python library *dilithium-py* [43]. This library is written for learning and is not meant for production use. We use it only to measure the cost of the signing steps.

B. METRICS

We compare the quarantine status of each node with its true label. We report two detection measures:

- Cumulative: a node counts as flagged if it was quarantined in any round of the run. This tells whether the node was ever caught.
- Final state: a node counts as flagged only if it is quarantined at round 100. This tells whether the node is still excluded at the end. We use it as the main measure.

From both we compute the detection rate $DR = TP/(TP+FN)$, the false-positive rate $FPR = FP/(FP+TN)$, the precision $P = TP/(TP+FP)$, and $F1 = 2PDR/(P + DR)$. We also report the share of honest nodes that were quarantined at least once. Where a table gives one value for “detection”, it is the mean over the four attacks that corrupt data (false data, faulty sensor, on-off, and collusion). In the bad-mouthing and no-attack scenarios, nothing can be detected, so only false positives matter. Standard deviations are population standard deviations over the five seeds. The aggregation error is the mean absolute difference between the aggregate D_j and the mean of the honest readings at the same gateway and round. It is computed

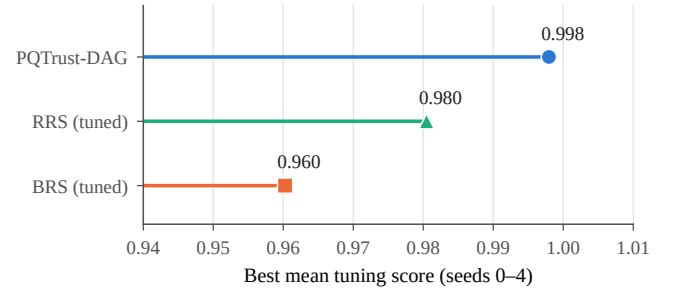


FIGURE 3. Best mean tuning score of each method over the five tuning seeds (0–4), using the objective of Eq. 11. A score of 1 is perfect.

over gateway-rounds that contain at least one malicious sensor.

C. TUNING PROTOCOL

We tuned every method on a smaller network (500 sensors, 50 gateways, 80 rounds) with seeds 0–4. We then tested on a larger network (1,000 sensors, 100 gateways, 100 rounds) with seeds 42–46. No test seed was used for tuning. PQTrust-DAG, BRS, and RRS use the same objective (Eq. 11), which uses final-state results. The naive baseline is not tuned; it uses fixed values.

$$\text{Score} = 0.35 \overline{DR} + 0.25 \min(DR) + 0.15 \overline{F1} + 0.10 (1 - \overline{FPR}) + 0.15 (1 - FPR_{bm}). \quad (11)$$

Here the mean and the minimum are taken over the four data-corrupting attacks. FPR_{bm} is the false-positive rate under bad-mouthing.

For PQTrust-DAG, the search used a grid. The weights (w_R, w_C, w_H, w_M) took values on a 0.2 grid that sums to 1. The memory m took values 0.7, 0.8, and 0.9. The quarantine threshold b_R took 0.40, 0.45, and 0.50. The gap $b_Q - b_R$ took 0.05 and 0.15. The constant z took 0.5 and 1.0. After duplicate settings were removed, 1,206 settings remained. In stage 1, every setting ran on seed 0. In stage 2, the 50 best settings and two reference settings that use all four signals (one is our earlier hand-chosen setting) ran on all five tuning seeds. Only 16 of these 52 settings use the reliability or peer-report signal, because most of the best settings of stage 1 had $w_R = w_M = 0$. Grid search is easy to repeat, although random search can be more efficient [44].

Figure 3 shows the best mean score of each method over the five tuning seeds. The chosen PQTrust-DAG setting is in Table 4. The best tuned BRS setting scored 0.960, and the best tuned RRS setting scored 0.980. The settings of the baselines are given in Section V-D.

Four limits apply to the search. First, for m , the gap, and z , the chosen value is an end value of the grid, and the chosen weights ($w_C = 0.2, w_H = 0.8$) are next to the grid edge, because $w_H = 1$ leaves no consistency signal. The score rose with w_H , so values of w_C between 0 and 0.2 were not tested, and a wider search could find

TABLE 4. Settings of PQTrust-DAG used in all test runs

Parameter	Value
Weights (w_R, w_C, w_H, w_M)	(0, 0.2, 0.8, 0)
History memory m	0.7
Thresholds (b_R, b_Q, b_A)	(0.45, 0.60, 0.75)
Consistency weights (k_1, k_2, k_3)	(0.3, 0.4, 0.3)
Check scales ($s_{temp}, s_{spatial}$)	(20, 15)
Peer-report constant z	1.0
Observers per node and round; vote limit	3; 40
Weight factor of Cautious nodes	0.5
Reliability window	100 transmissions

better values. Second, the check scales of 20 and 15 and the vote limit of 40 were not tuned. For the scales, we repeated the search with scales twice and three times as large and re-tuned the weights and thresholds. The best scores were 0.996 and 0.997, against 0.998 for the chosen setting, so a wider scale gave no gain. Third, the PQTrust-DAG grid has 1,206 settings, while the grids of the baselines have 32 and 53 settings (Section V-D), so PQTrust-DAG had more room to be tuned. Fourth, only 16 of the 52 settings in stage 2 use reliability or peer reports (Section V-I).

D. BASELINES

We compare PQTrust-DAG with three baselines. Two are published reputation systems, which we rebuilt from the equations of their papers. The third is a simple scheme that we designed as a reference point.

BRS [41]. Each node has a positive count r and a negative count s . Every round, both counts are multiplied by a forgetting factor λ , and the gateway adds its own observation. Votes from observers are added after they are discounted by the reputation of the observer. The score is $(r + 1)/(r + s + 2)$, and a node is flagged when the score is below a threshold τ .

RRS [42]. Each node has a rating built from the gateway's own observations and from the observers' records. A record from an observer is used only if the observer is trusted or if it agrees with the current rating (a deviation test with limit d). The trust in an observer drops when it fails the test. A node is flagged when its rating of misbehavior reaches r . The symbols u and v are forgetting factors, d is the deviation limit, and w and t are a weight and a trust limit; we use the paper's values $w = 0.1$ and $t = 0.25$.

Naive reputation. A moving average of transmission validity, unweighted majority votes from observers, and one threshold for entering and leaving quarantine (no hysteresis).

BRS and RRS get the same evidence as PQTrust-DAG: the same votes from the observers, and the gateway's own observation. The gateway's own observation is "misbehavior" if the reading is outside $[0, 120]$ or more than 40 away from the average of the sampled neighbors. The original papers do not say how misbehavior is

detected, so this is our choice. The naive baseline uses only the range check and the votes. PQTrust-DAG also uses graded consistency scores, which the baselines do not have. The comparison therefore tests the whole design and not only the update rule. The baselines have no tiers, so a flagged node is simply excluded, and the gateway averages the readings of all other nodes.

We tuned BRS and RRS with the objective of Eq. 11 on seeds 0–4. For BRS, we tried $\lambda \in \{0.7, 0.8, 0.9, 0.95\}$ and eight values of τ from 0.4 to 0.9 (32 settings). The best was $\lambda = 0.95$ and $\tau = 0.75$. For RRS, we tried 53 combinations of $u = v \in \{0.6, \dots, 0.95\}$, $d \in \{0.2, 0.3, 0.4\}$, and $r \in \{0.1, \dots, 0.5\}$, and we extended the grid twice when the best value was at its edge. The best was $u = v = 0.9$, $d = 0.3$, and $r = 0.2$. Because $\lambda = 0.95$ was the largest forgetting factor in the BRS grid, we also tried $\lambda \in \{0.97, 0.98, 0.99\}$ with five values of τ . None scored higher (the best of these scored 0.942, against 0.960).

E. MAIN COMPARISON

Table 5 shows the results on the five test seeds. Figure 4 shows the final-state detection rate for each attack, and Figure 5 shows the share of honest nodes that were quarantined at least once.

At the end of the run, PQTrust-DAG detects 99.7% of attackers. This is 8 missed attackers out of 3,000 attacker cases (150 attackers, four attacks, five seeds). All 8 ended in the Cautious tier. It wrongly quarantines fewer than 0.1% of honest nodes. BRS and RRS detect slightly more attackers (99.9% and 100%). But they wrongly quarantine 1.6% and 2.4% of honest nodes at the end of the run, so their F1 scores are lower (0.956 and 0.938, against 0.998). Figure 5 shows a much larger difference. During the run, BRS quarantines 97% of honest nodes at least once, and RRS quarantines all of them. PQTrust-DAG quarantines fewer than 0.1%. This follows from the binary evidence rule that BRS and RRS receive: an honest reading is sometimes more than 40 away from the neighbor average, and the tuned thresholds react to a few such readings. The naive baseline has fewer false alarms than BRS and RRS but misses more attackers (86.6% detected). Its detection is lowest for faulty sensors (71%).

Under bad-mouthing, the final-state false-positive rate of BRS is 16.2%, against 0 for PQTrust-DAG and RRS. The result for PQTrust-DAG needs a careful reading. The chosen setting gives the peer reports a weight of zero ($w_M = 0$), so bad-mouthing cannot reach the trust score. It is safe because it ignores the reports, and not because it weights them well (Section V-I).

F. TRUST OVER TIME

Figures 6 and 7 show the mean trust of honest and malicious nodes in one run (seed 42). Under false data, the trust of malicious nodes drops from 0.58 to below the

TABLE 5. Test results (seeds 42–46, mean $\pm$ std over seeds). Detection, false-positive rate (FPR), and F1 are final-state values averaged over the four data-corrupting attacks. “Honest ever quarantined” is the share of honest nodes quarantined at least once. The last column is the final-state FPR under bad-mouthing.

Method	Detection rate	False-positive rate	F1 score	Honest ever quarantined	Bad-mouthing FPR
PQTrust-DAG	0.997 ± 0.003	$<0.001 \pm 0.000$	0.998 ± 0.002	<0.001	0
BRS (tuned)	0.999 ± 0.002	0.016 ± 0.003	0.956 ± 0.007	0.974	0.162
RRS (tuned)	1.000 ± 0.001	0.024 ± 0.003	0.938 ± 0.007	1.000	0
Naive reputation	0.866 ± 0.011	0.006 ± 0.001	0.909 ± 0.010	0.062	0.021

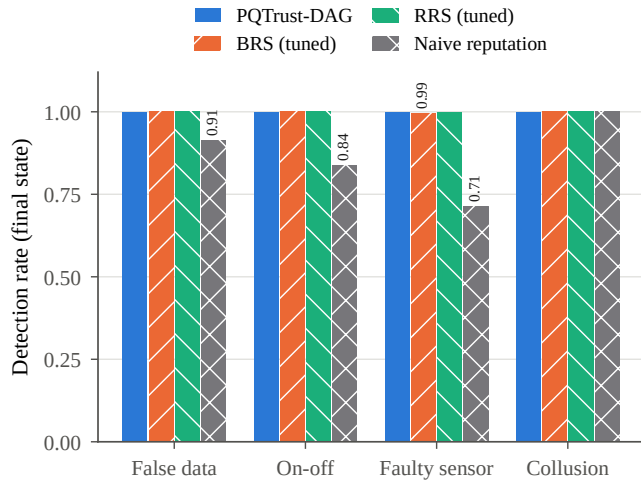


FIGURE 4. Final-state detection rate for each attack (test seeds 42–46). Only values below 0.995 are labeled; the other bars are at 0.997 to 1.000.

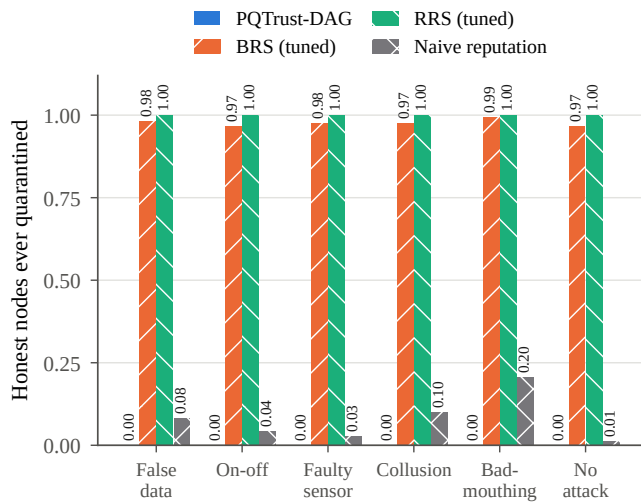


FIGURE 5. Share of honest nodes that were quarantined at least once during the run, for each scenario (test seeds 42–46). Lower is better.

quarantine threshold of 0.45 within the first 5 rounds, and it stays near 0.3 for the rest of the run. Honest nodes settle at about 0.62.

Under the on-off attack, the malicious nodes look like honest nodes for about 60 rounds and have about the same mean trust. After they switch to false data, their trust falls below 0.45 at about round 70 and reaches 0.34

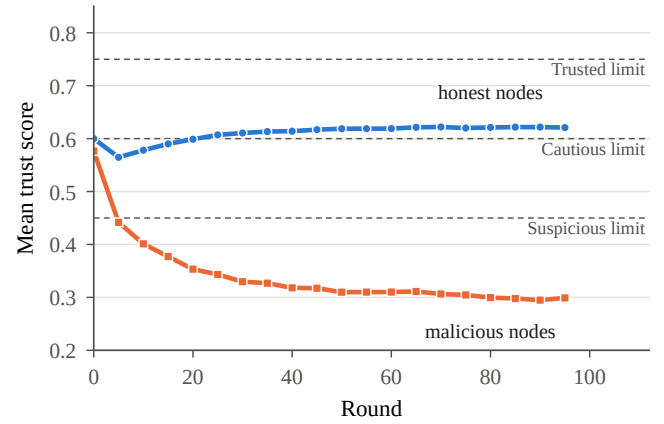


FIGURE 6. Mean trust of honest and malicious nodes under false-data injection (seed 42). Dashed lines mark the tier limits.

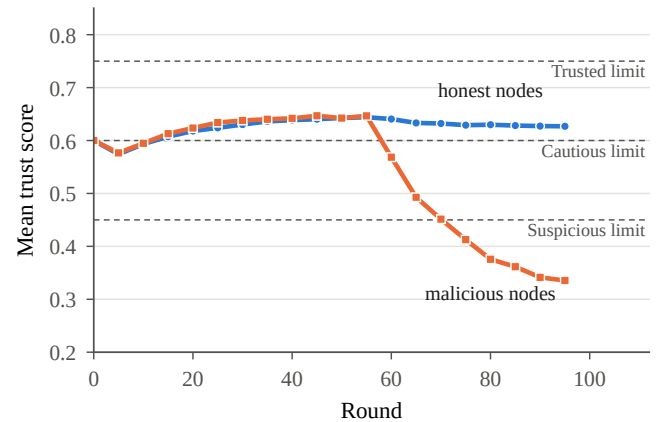


FIGURE 7. Mean trust of honest and malicious nodes under the on-off attack (seed 42). The attackers switch to false data after about round 60.

at round 95. A delay of this size is expected, because the history signal remembers the earlier, good trust of the node.

G. TIER DISTRIBUTION

Figure 8 shows the tiers of all nodes at round 100, pooled over the four data-corrupting attacks and the five test seeds. Of the malicious nodes, 99.7% are Quarantined and 0.3% are Cautious. Of the honest nodes, 72.3% are Cautious, 27.5% are Suspicious, 0.15% are Trusted, and 0.04% are Quarantined. In the no-attack scenario, the

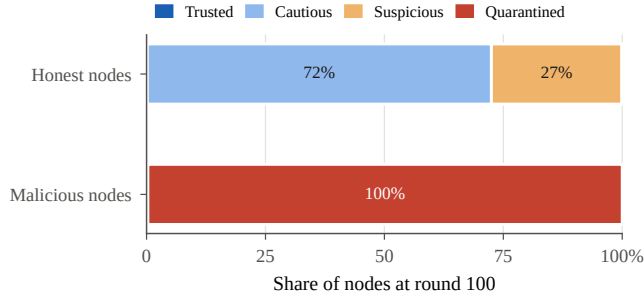


FIGURE 8. Tier of honest and malicious nodes at round 100, pooled over the four data-corrupting attacks and seeds 42–46. Labels are shown for shares above 7%.

TABLE 6. Ablation results (test seeds 42–46; final state, averaged over the four data-corrupting attacks). DR: detection rate. FPR: false-positive rate. Ever: share of honest nodes quarantined at least once. Agg. error: aggregation error.

Variant	DR	FPR	Ever	Agg. error
Full model	0.997	<0.001	<0.001	4.25
No hysteresis	0.953	0	<0.001	4.27
No history ($w_H=0$)	0.825	0.215	0.999	6.17
History of R (old design)	0.150	0	0	2.15
Reliability only	0.082	0	0	3.68
No consistency ($w_C=0$)	0	0	0	4.87
Cautious at full weight	0.997	<0.001	<0.001	4.25
With corroboration	0.997	<0.001	<0.001	3.46

honest nodes are more often Cautious (88.5%) and less often Suspicious (11.3%). The bad readings of the attackers lower the consistency scores of honest neighbors, because the neighbor average includes the bad readings.

This is a weak point of the current setting. Almost no honest node reaches the Trusted tier, because honest readings in our simulation are noisy, and the trust of honest nodes settles at about 0.6 to 0.65, below the Trusted limit of 0.75. About a quarter of the honest nodes are held out of the aggregate as Suspicious when attackers are present.

H. ABLATION STUDY

Table 6 and Figure 9 show what happens when one part of the model is changed. All variants use the chosen setting unless noted.

Without the consistency signal ($w_C = 0$), nothing is detected, because the score then stays at its start value of 0.5. This shows only that some signal based on the data is needed. The history signal is also essential: with $w_H = 0$, detection falls to 0.825, the false-positive rate rises to 0.215, and 99.9% of honest nodes are quarantined at least once. If the history averages the reliability signal, as in our first design, detection is only 0.150, and a score built on reliability alone reaches 0.082. The reliability signal helps little here, because it counts only readings outside the physical range $[0, 120]$, and many bad readings are still inside that range. Without

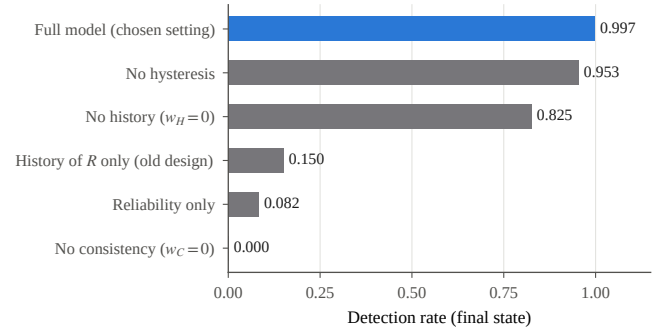


FIGURE 9. Final-state detection rate of the full model and of five ablations (test seeds 42–46).

TABLE 7. Settings that use peer reports or reliability (test seeds 42–46; final state, averaged over the four data-corrupting attacks). DR, FPR, and Ever are as in Table 6. Ever (BM) is the share of honest nodes quarantined at least once under bad-mouthing.

Setting	DR	FPR	Ever	Ever (BM)
Chosen setting	0.997	<0.001	<0.001	0
Peer reports ($w_M=0.2$)	0.941	0.001	0.155	0.169
Reliability ($w_R=0.2$)	0.895	<0.001	0.005	0.005
All four signals	0.711	<0.001	0.015	0.015
Reports removed	0.997	0.015	0.293	0.210
Votes unweighted	0.969	0.011	0.304	0.400

hysteresis, detection falls to 0.953. The loss is largest for faulty sensors (0.879) and on-off attackers (0.948), likely because their trust moves up and down near the threshold. Giving Cautious nodes full weight changes nothing, probably because almost every node in the aggregate is Cautious and all of them get the same factor.

I. PEER REPORTS AND RELIABILITY

Table 7 shows what happens when we give weight to the peer-report signal or the reliability signal. For each case, we took the best setting of the stage-2 search that has this signal: (w_R, w_C, w_H, w_M) = (0, 0.2, 0.6, 0.2) for peer reports, (0.2, 0.6, 0.2, 0) for reliability, and (0.2, 0.4, 0.2, 0.2) for all four signals. The values of m , b_R , and z also come from the search and differ between rows. The last two rows use the peer-report setting with the reports removed (the other weights are scaled to sum to 1) and with votes that are not weighted by trust.

Neither signal improves the tuning score of the best setting. With $w_M = 0.2$, detection falls to 0.941, and 15.5% of honest nodes are quarantined at least once. With $w_R = 0.2$, detection falls to 0.895. With all four signals, detection falls to 0.711. The peer reports do lower false alarms in the peer-report setting. When they are removed, the false-positive rate rises from 0.001 to 0.015, but detection also rises, from 0.941 to 0.997. So the reports trade detection for fewer false alarms. Weighting the votes by trust helps further. Compared

TABLE 8. Signed end-to-end runs (seed 42, final model, 100,000 signatures and 10,000 DAG transactions per run). Signing times were measured while three runs shared the machine. DR / FPR are final-state values.

Scenario	Sign (ms)	Total (min)	Finalized	DR / FPR
False data	52.8	88	8,104	1.000 / 0.000
On-off	52.8	88	8,104	1.000 / 0.000
Faulty sensor	52.7	88	8,104	1.000 / 0.000
Collusion	49.7	83	8,104	1.000 / 0.000
Bad-mouthing	49.5	83	8,104	n/a / 0.000
No attack	49.6	83	8,104	n/a / 0.000

with unweighted votes in the same setting, trust weighting lowers the share of honest nodes that are quarantined at least once under bad-mouthing from 0.400 to 0.169, at a small cost in detection (0.969 for unweighted votes against 0.941).

These results have limits. Only 16 of the 52 settings of stage 2 use reliability or peer reports, and they were picked from stage 1 with one seed, so a larger search could find a better setting that uses them. One possible reason for the weak effect of the reports is that the votes in our simulation test the same thing as the consistency signal (the distance from the neighbor average). We did not test this. A setting where observers see information that the gateway does not have (for example, direct views of the road) could give a different result.

J. AGGREGATION ACCURACY

Figure 10 shows the aggregation error. Over the four data-corrupting attacks, the plain mean of all readings has an error of 8.61. PQTrust-DAG reduces it to 4.25, which is about half. With the optional corroboration rule, the error falls to 3.46. BRS gives 4.25 and RRS gives 6.61. The naive baseline gives the smallest error, 1.90.

So PQTrust-DAG is not the best method for this measure. Two likely reasons are as follows. First, PQTrust-DAG holds back honest Suspicious nodes, so it uses fewer honest readings. Second, the naive baseline flags few honest nodes, so its aggregate uses most honest readings. In the no-attack scenario, the error of PQTrust-DAG is 3.73, while the plain mean has an error of 2.18. Trust weighting pays off only when attackers are present.

K. SIGNED END-TO-END RUNS

We ran all six scenarios at full scale (1,000 sensors, 100 gateways, 100 rounds, seed 42) with ML-DSA-65 signing and DAG commits turned on together. Each run signs 100,000 trust updates, one per sensor and round, with one shared ledger key. The runs were done in a Linux virtual machine on an Apple-silicon Mac with 4 cores, in short pieces with saved checkpoints. Table 8 shows the results.

All six runs gave exactly the same detection results, tiers, and DAG counts as unsigned runs of the same

program (with the DAG switched on) on the same seed. Turning on signing changes only the running time. The average time per signature was 49.5 to 52.8 ms while three runs shared the machine, which gives 83 to 88 minutes of signing per run. About 81% of the 10,000 DAG transactions were finalized in every scenario (8,104 of 10,000). The finalization rate is the same in all six scenarios, so in these runs it does not depend on the behavior of the nodes.

We also timed signing on an otherwise idle machine, with 1,000 signatures. Signing took 33.5 ms on average (median 26.5 ms, 95th percentile 83.2 ms). Verification took 6.2 ms on average. A signature has 3,309 bytes and a public key has 1,952 bytes. All 1,000 signatures verified. Entries that were given another node ID, another time stamp, or an altered hash were rejected. The six full runs used an earlier message format that signed only the two hashes. After the runs, we changed the signed message to include the node ID and the time (Algorithm 3). The benchmark uses the final format. The extra bytes did not change the signing time in a measurable way.

These timings come from a pure-Python library in a virtual machine. An optimized C library would be much faster, so they show the order of cost and not the best case.

VI. DISCUSSION

A. WHAT THE RESULTS SHOW

The results support a narrow claim. A trust score that smooths a consistency check over many rounds, and that remembers its own earlier value, detects almost all attackers at the end of a run with very few false alarms. Two published reputation systems, with the same evidence and tuning, detect about the same number of attackers but raise many more false alarms. Adding the peer-report signal or the reliability signal lowered detection, although the peer reports reduced false alarms in one setting. The working model is therefore close to a smoothed consistency score with a quarantine gap. The design still includes both signals, because they may matter in settings that our simulation does not cover.

B. LIMITS OF THE SIMULATION

The honest readings in our simulation are independent draws from a normal distribution, so honest readings are noisy. This keeps honest trust near 0.6 to 0.65 and keeps almost all honest nodes out of the Trusted tier. Real traffic data are more regular, and the thresholds would need to be tuned again. The attacks are simple. Attackers do not try to copy the statistics of their neighbors or to stay just inside the vote limit. The voting rule of the observers and the limit of 40 were set by us. Results with a different rule may differ. We used 15% attackers and one network layout per seed, and did not test other densities. We report the mean and standard

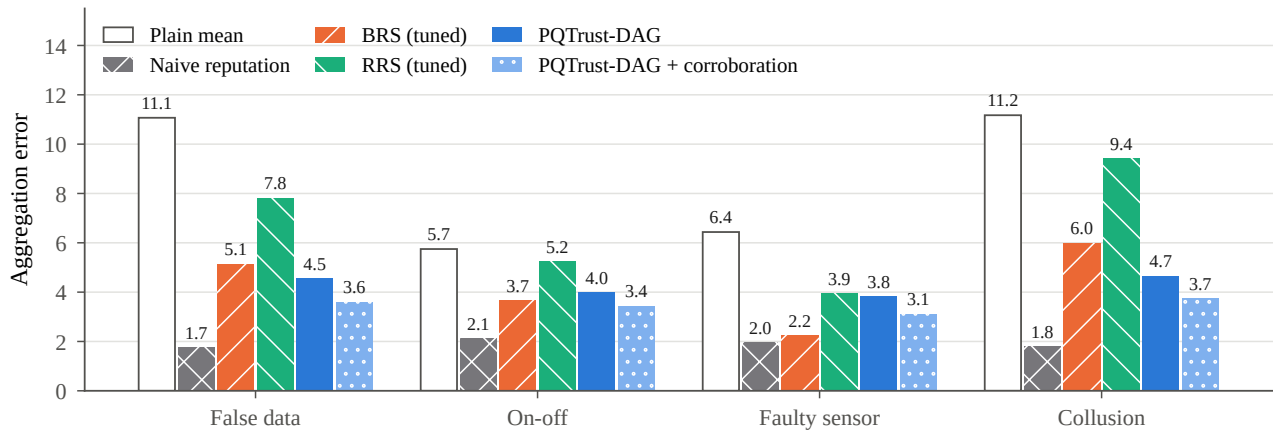


FIGURE 10. Aggregation error for each attack (test seeds 42–46). The error is the mean absolute difference between the aggregate and the mean of the honest readings. Lower is better.

deviation over five test seeds, but the simulation has no confidence intervals beyond that, and the signed runs use one seed.

C. FAIRNESS OF THE COMPARISON

We tried to give every method the same chance. BRS, RRS, and PQTrust-DAG use the same objective, the same network, and the same tuning seeds, and none was tuned on the test seeds. BRS and RRS get the same evidence as the observers of PQTrust-DAG. The naive baseline is not tuned. Still, two points favor PQTrust-DAG. It has graded consistency scores, and it was tuned over a larger grid (1,206 settings against 32 and 53). The baselines were built for open networks and were not designed for sensor data, and their papers leave the detection of misbehavior open. A different choice of evidence could change their results. Our claim is therefore limited to this setting.

D. PARAMETER TUNING IN PRACTICE

Anyone who deploys the system should tune ($w_R, w_C, w_H, w_M, m, b_R, b_Q$) again for their own share of attackers, sensor noise, and network density. The tuning objective rewards a low false-positive rate under bad-mouthing. Setting $w_M = 0$ reaches this by ignoring the peer reports, and not by defending against bad-mouthing. This is what happened in our search. A future search should include attackers that make peer reports useful, for example observers that have information that the gateway lacks.

E. SECURITY SCOPE

1) Sybil Identities

Sybil identities are handled by the identity layer. BC_ID is permissioned, so every new identity must be registered through a trusted authority. The trust layer adds only that a new node starts at neutral trust (0.5) and must earn trust before its data counts much.

2) Cryptographic Guarantees

BC_Trust uses ML-DSA-65 signatures. This parameter set targets NIST security category 3 [25], the same category as the Kyber-768 (ML-KEM-768) key exchange of the base architecture [7], [24]. We rely on the published security proofs of these schemes and do not prove them again. The library that we used is for research and learning, and it is not protected against side-channel attacks [43]. A real deployment needs a hardened library.

3) Formal Properties

Trust integrity, detection of malicious nodes, resistance to false reports, and auditability are design goals of PQTrust-DAG. They are not proven properties. The simulation results agree with these goals in the tested settings but do not prove them.

F. COLD START

A new vehicle has no history. It could start at neutral trust (as in our simulation), receive extra trust that the manufacturer certifies at registration, stay in the Cautious tier until it has a record, or rely on a certificate-based trust hierarchy above the behavioral score.

G. PRIVACY

Checking behavior for trust is a form of surveillance of how vehicles drive and report. BC_Trust limits this by storing only hashes and signatures, and never raw behavior data. The design also needs changing pseudonyms and differential privacy, which are not implemented (Section III-C). Lightweight pseudonym authentication schemes such as FC-PA [45], which needs only one elliptic-curve scalar multiplication to authenticate a message, are a practical starting point for changing identifiers.

VII. CONCLUSION

This paper presented PQTrust-DAG, a behavioral trust layer for post-quantum, DAG-based vehicular sensor networks. It addresses a gap that authentication cannot close: a vehicle with valid credentials can still send false data. We built the trust layer, trust-weighted aggregation, DAG commits, and a BC_Trust ledger signed with ML-DSA-65, and we tested them on 1,000 sensors and 100 gateways in six scenarios.

We draw four conclusions. First, a trust score that smooths a consistency check and remembers its own past detects 99.7% of attackers at the end of a run, with fewer than 0.1% false positives. Second, two published reputation systems, rebuilt from their papers and tuned in the same way, detect slightly more attackers but, with our evidence rule, quarantine 1.6% and 2.4% of honest nodes at the end of a run. Third, the history and consistency terms carry the result, while adding the peer-report or reliability terms lowered detection; and honest nodes rarely reach the Trusted tier. Fourth, signing 100,000 trust updates took 83 to 88 minutes per run with a pure-Python library, and 81% of DAG transactions were finalized, without any change in the detection results. All results come from simulation.

Future work has three parts. For the system, we will build the missing parts: PoPQW consensus and threshold-signature finality, building on quantum-sampling proof-of-work [46]; the extended transaction format; the time stamp check; signed peer reports; changing pseudonyms; and differential privacy. We will also measure DAG throughput at scale with DAG-ledger performance models [21]. In addition, we will use an optimized signature library. For the evaluation, we will use realistic vehicle movement and traffic data, for example in the VEINS framework, whose use in VANET security research is reviewed in [47]. We will test attackers that copy their neighbors' statistics, other attacker shares and network densities, and settings where peer reports carry new information. In the longer term, we plan to study a learned trust score, for example one trained with federated learning, building on DAG-blockchain federated learning [48] and blockchain-based federated traffic prediction [49].

ACKNOWLEDGMENT

The authors used Claude for language editing (grammar, style, and readability). After using this tool, they reviewed and edited all content and take full responsibility for the manuscript.

REFERENCES

- [1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, IEEE, 1994, pp. 124–134.
- [2] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput.*, 1996, pp. 212–219.
- [3] G. Liao, H. Ding, and Y. Lei, "IORT-DAG: A real-time DAG-based blockchain with implicit ordering," *Future Gener. Comput. Syst.*, vol. 175, Art. no. 108041, 2026, doi: 10.1016/j.future.2025.108041.
- [4] Y. Bai, S. Lee, and S.-H. Seo, "A survey on directed acyclic graph-based blockchain in smart mobility," *Sensors*, vol. 25, no. 4, p. 1108, 2025.
- [5] M. Raikwar, N. Polyanskii, and S. Müller, "SoK: DAG-based consensus protocols," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, 2024, pp. 1–18, doi: 10.1109/ICBC59979.2024.10634358.
- [6] A. Mahdi and F. Rabee, "DAGchains: An improved blockchain structure based on directed acyclic graph construction and distributed mining," *Mesopotamian J. Cybersecur.*, vol. 5, no. 2, pp. 375–394, 2025, doi: 10.58496/MJCS/2025/024.
- [7] S. Ahmed, N. Tariq, F. A. Khan, and S. H. Chauhdary, "Post-quantum DAG-based blockchain for securing vehicular sensor networks," *IEEE Trans. Consumer Electron.*, vol. 72, no. 3, pp. 5791–5799, Aug. 2026, doi: 10.1109/TCE.2026.3688933.
- [8] Q. Xia, E. B. Sifah, A. Smahi, S. Amofa, and X. Zhang, "BBDS: Blockchain-based data sharing for electronic medical records in cloud environments," *Information*, vol. 8, no. 2, Art. no. 44, 2017, doi: 10.3390/info8020044.
- [9] Q. Xia, E. B. Sifah, K. O. Asamoah, J. Gao, X. Du, and M. Guizani, "MedShare: Trust-less medical data sharing among cloud service providers via blockchain," *IEEE Access*, vol. 5, pp. 14757–14767, 2017, doi: 10.1109/ACCESS.2017.2730843.
- [10] J. Kang, R. Yu, X. Huang, M. Wu, S. Maharjan, S. Xie, and Y. Zhang, "Blockchain for secure and efficient data sharing in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4660–4670, 2019.
- [11] X. Zhang and X. Chen, "Data security sharing and storage based on a consortium blockchain in a vehicular ad-hoc network," *IEEE Access*, vol. 7, pp. 58241–58254, 2019, doi: 10.1109/ACCESS.2018.2890736.
- [12] K. Fan, Q. Pan, K. Zhang, Y. Bai, S. Sun, H. Li, and Y. Yang, "A secure and verifiable data sharing scheme based on blockchain in vehicular social networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 6, pp. 5826–5835, 2020.
- [13] M. Cui, D. Han, H. Liu, K.-C. Li, M. Tang, C.-C. Chang, F. Ayaz, Z. Sheng, and Y. L. Guan, "Secure data sharing for consortium blockchain-enabled vehicular social networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 12, pp. 19682–19695, 2024.
- [14] A. Rahman, M. Z. A. Eidmum, D. Kundu, M. Hossain, M. T. A. Tashrif, M. A. Karim, and M. J. Islam, "DistB-VNET: Distributed cluster-based blockchain vehicular ad-hoc networks through SDN-NFV for smart city," 2024, arXiv:2412.04222.
- [15] E. Vieira, J. Almeida, J. Ferreira, and P. C. Bartolomeu, "Enabling seamless data security, consensus, and trading in vehicular networks," *IEEE Trans. Intell. Veh.*, vol. 9, no. 10, pp. 6716–6737, Oct. 2024.
- [16] J. de Anda-Suárez et al., "An integrated blockchain framework for secure autonomous vehicle communication system," *Information*, vol. 16, no. 7, p. 557, 2025.
- [17] D. Das, K. Dasgupta, and U. Biswas, "A secure blockchain-enabled vehicle identity management framework for intelligent transportation systems," *Comput. Elect. Eng.*, vol. 105, p. 108535, 2023.
- [18] M. Zrikem, I. Hasnaoui, and R. Ellassali, "Vehicle-to-blockchain (V2B) communication: Integrating blockchain into V2X and IoT for next-generation transportation systems," *Electronics*, vol. 12, no. 16, p. 3377, 2023.
- [19] G. Bendiab, A. Hameurlaine, G. Germanos, N. Kolokotronis, and S. Shiaeles, "Autonomous vehicles security: Challenges and solutions using blockchain and artificial intelligence," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 3614–3637, Apr. 2023.
- [20] Q. Xie, Z. Ding, W. Tang, D. He, and X. Tan, "Provable secure and lightweight blockchain-based V2I handover authentication and V2V broadcast protocol for VANETs," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 15200–15212, Dec. 2023.

- [21] J. Shi, X.-Y. Bai, W.-Z. Zhang, P.-L. Li, K.-D. Wu, G.-L. Yang, and M.-T. Zhang, "Performance modeling and testing of DAG-based distributed ledger systems," *J. Comput. Sci. Technol.*, vol. 40, no. 6, pp. 1593–1607, Nov. 2025, doi: 10.1007/s11390-025-4616-8.
- [22] V. Hassija, V. Chamola, S. Garg, D. N. G. Krishna, G. Kaddoum, and D. N. K. Jayakody, "A blockchain-based framework for lightweight data sharing and energy trading in V2G network," *IEEE Trans. Veh. Technol.*, vol. 69, no. 6, pp. 5799–5812, 2020.
- [23] H. Chai, S. Leng, F. Wu, and J. He, "Secure and efficient blockchain-based knowledge sharing for intelligent connected vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 14620–14631, 2021.
- [24] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," *Federal Information Processing Standards Publication (FIPS) 203*, Aug. 2024, doi: 10.6028/NIST.FIPS.203.
- [25] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," *Federal Information Processing Standards Publication (FIPS) 204*, Aug. 2024, doi: 10.6028/NIST.FIPS.204.
- [26] National Institute of Standards and Technology, "Stateless hash-based digital signature standard," *Federal Information Processing Standards Publication (FIPS) 205*, Aug. 2024, doi: 10.6028/NIST.FIPS.205.
- [27] G. Alagic et al., "Status report on the fourth round of the NIST post-quantum cryptography standardization process," NIST, Gaithersburg, MD, USA, NIST IR 8545, Mar. 2025, doi: 10.6028/NIST.IR.8545.
- [28] S. Xu, T. Wang, A. Sun, Y. Tong, Z. Ren, R. Zhu, and H. H. Song, "Post-quantum anonymous, traceable and linkable authentication scheme based on blockchain for intelligent vehicular transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 9, pp. 12108–12119, Sep. 2024.
- [29] S. Son, D. Kwon, S. Lee, Y. Jeon, A. K. Das, and Y. Park, "Design of secure and lightweight authentication scheme for UAV-enabled intelligent transportation systems using blockchain and PUF," *IEEE Access*, vol. 11, pp. 60240–60253, 2023, doi: 10.1109/ACCESS.2023.3286016.
- [30] S. H. Chauhdary, T. Ma, L. Jiang, F. A. Khan, A. K. Das, and S. A. Chaudhry, "PUF-ILM: A PUF-enabled authentication scheme for Internet of Vehicles using improved logical mapping," *IEEE Internet Things J.*, vol. 13, no. 7, pp. 13787–13800, Apr. 2026, doi: 10.1109/JIOT.2026.3656432.
- [31] J. Chen, W. Gan, M. Hu, and C.-M. Chen, "On the construction of a post-quantum blockchain for smart city," *J. Inf. Secur. Appl.*, vol. 58, Art. no. 102780, 2021, doi: 10.1016/j.jisa.2021.102780.
- [32] P. K. Sandanamudi, N. Agrawal, N. Tripathi, and P. K. B. N., "Securing UAV communications: A comparative performance analysis of post-quantum cryptographic techniques," in *Proc. 17th Int. Conf. Commun. Syst. Netw. (COMSNETS)*, 2025, pp. 1096–1101, doi: 10.1109/COMSNETS63942.2025.10885608.
- [33] J. H.-y. Chiang et al., "Post-quantum threshold ring signature applications from Vole-in-the-Head," in *Proc. ACM Conf. Comput. Commun. Secur. (CCS)*, 2025, pp. 4664–4678.
- [34] A. Hbaieb, S. Ayed, and L. Chaari, "A survey of trust management in the Internet of Vehicles," *Comput. Netw.*, vol. 203, Art. no. 108558, 2022, doi: 10.1016/j.comnet.2021.108558.
- [35] H. Che, Y. Duan, C. Li, and L. Yu, "On trust management in vehicular ad hoc networks: A comprehensive review," *Front. Internet Things*, vol. 1, Art. no. 995233, 2022, doi: 10.3389/friot.2022.995233.
- [36] H. Amari, Z. Abou El Houda, L. Khokhi, and L. Hadrich Belguith, "Trust management in vehicular ad-hoc networks: Extensive survey," *IEEE Access*, vol. 11, pp. 47659–47680, 2023, doi: 10.1109/ACCESS.2023.3268991.
- [37] Y. Liu, J. Wang, Z. Yan, Z. Wan, and R. Jäntti, "A survey on blockchain-based trust management for Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5898–5922, Apr. 2023.
- [38] S. Malik, V. Dedeoglu, S. S. Kanhere, and R. Jurdak, "TrustChain: Trust management in blockchain and IoT supported supply chains," in *Proc. IEEE Int. Conf. Blockchain (Blockchain)*, 2019, pp. 184–193, doi: 10.1109/Blockchain.2019.00032.
- [39] T. Ahmad, S. Khan, N. Nigar, E. M. Obaid, and S. Naaz, "Quantum reservoir-based intrusion detection for edge-IoT systems," *IEEE Trans. Consumer Electron.*, vol. 71, no. 4, pp. 11269–11278, Nov. 2025.
- [40] M. Fu, P. Wang, M. Liu, Z. Zhang, and X. Zhou, "IoV-BERT-IDS: Hybrid network intrusion detection system in IoV using large language models," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 1909–1921, Feb. 2025.
- [41] A. Jøsang and R. Ismail, "The Beta Reputation System," in *Proc. 15th Bled Electron. Commerce Conf., Bled, Slovenia, Jun. 2002*.
- [42] S. Buchegger and J.-Y. Le Boudec, "A robust reputation system for P2P and mobile ad-hoc networks," in *Proc. 2nd Workshop Econ. Peer-to-Peer Syst. (P2PEcon)*, Cambridge, MA, USA, Jun. 2004.
- [43] G. Pope, "dillithium-py: A pure-Python implementation of ML-DSA (FIPS 204)," version 1.4.0, GitHub repository. [Online]. Available: <https://github.com/GiacomoPope/dilithium-py>. Accessed: Oct. 4, 2026.
- [44] J. Bergstra and Y. Bengio, "Random search for hyperparameter optimization," *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, 2012.
- [45] B. A. Mohammed, M. A. Al-Shareeda, S. Manickam, Z. G. Al-Mekhlafi, A. Alreshidi, M. Alazmi, J. S. Alshudukhi, and M. Alsaffar, "FC-PA: Fog computing-based pseudonym authentication scheme in 5G-enabled vehicular networks," *IEEE Access*, vol. 11, pp. 18571–18581, 2023, doi: 10.1109/ACCESS.2023.3247222.
- [46] D. Singh et al., "Proof-of-work consensus by quantum sampling," *Quantum Sci. Technol.*, vol. 10, no. 2, p. 025020, 2025.
- [47] M. A. Al-Shareeda and S. Manickam, "A systematic literature review on security of vehicular ad-hoc network (VANET) based on VEINS framework," *IEEE Access*, vol. 11, pp. 46218–46228, 2023, doi: 10.1109/ACCESS.2023.3274774.
- [48] S. Xiao, X. Huang, X. Deng, B. Cao, and Q. Chen, "DB-FL: DAG blockchain-enabled generalized federated dropout learning," *Digit. Commun. Netw.*, vol. 11, no. 3, pp. 886–897, 2025, doi: 10.1016/j.dcan.2024.09.005.
- [49] C. Meese, H. Chen, W. Li, D. Lee, H. Guo, C.-C. Shen, and M. Nejad, "Adaptive traffic prediction at the ITS edge with online models and blockchain-based federated learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 9, pp. 10725–10740, Sep. 2024.



L DILEEP SAI received the B.Tech. (Honors) degree in computer science and engineering from Koneru Lakshmaiah (KL) University, Vijayawada, India, in 2024, and is currently pursuing the M.Tech. degree in cybersecurity at SRM University-AP, Amaravathi, India. He interned at the Defence Research and Development Laboratory (DRDL), Hyderabad, working on vulnerability assessment. He holds the Certified Ethical Hacker (CEH) credential from EC-Council and is a Microsoft Azure Fundamentals and Oracle Cloud Infrastructure 2023 Certified Associate. His research interests include cybersecurity and network security, intrusion detection, digital forensics, zero-trust architectures for IoT, post-quantum cryptography, and blockchain-based security mechanisms.



UMA SANKARARAO VARRI received the M. Tech. degree from Jawaharlal Nehru Technological University, Kakinada, India, in 2015, and Ph.D. degree from National Institute of Technology-Warangal, Telangana, India. He is currently an Assistant Professor from the Department of Computer Science and Engineering, SRM University-AP, India. His research interests include security and privacy of data, information retrieval, quantum cryptography and blockchain.

...